Cache em VHDL

Lukas Ely[*], Gabriel de Menezes[†] e Emanuel Barros[‡]
Engenharia de Computação — PUCRS

1 Introdução

A cache, como uma memória de acesso rápido e alta velocidade, desempenha um papel crucial na determinação da eficiência geral do sistema. Sendo assim, este relatório apresenta a comparação entre modelos de teste desenvolvidos em VHDL, com o objetivo de comparar as eficiências baseados na hierarquia de memória. Para isso serão utilizados 3 modelos, a MP-0 (memória ideal sem atraso para retornar informações), a MP com atraso (memória RAM com 8 ciclos de atraso), e uma hierarquia com memória principal (com atraso) e uma cache L1.

A MP-0, essencialmente, é uma memória sem atraso, logo representa uma memória ideal. Esta, já tendo sido fornecida, será utilizada não apenas de base para os testes, mas também como base na implementação dos outros modelos.

2 Códigos em assembly e dumps

Para validar os modelos, foram elaborados programas em linguagem assembly MIPS utilizando o ambiente de desenvolvimento Mars. Posteriormente, esses códigos foram submetidos a simulação, utilizando o *dump* gerado como referência. Os códigos utilizados para teste foram:

1. media: retorna a média de um vetor
2. mtz_coluna: faz a leitura de uma matriz por coluna
3. mtz_linha: faz a leitura de uma matriz por linha
4. overflow: faz a verificação de um overflow em um registrador
5. bubblesort: faz a ordenação de um vetor a partir do algoritmo bubble sort

3 Implementação da MP com atraso

A MP com atraso representa uma memória principal com atraso fixo de 8 ciclos de relógio. Este atraso não pode ser implementado, logo ele será reporduzido a partir de um módulo de *delay*, que controla o tempo em que o processador ficará aguardando até que o dado esteja "pronto" (8 ciclos tenham se passado). Este controle é feito a partir de um sinal de *halt* (sinal de parada), que irá bloquear a execução dos estágios/instruções conseguintes do processador até que tenham se passado os 8 ciclos de *clock*.

4 Implementação da cache L1

A cache L1 vem como uma alternativa para solucionar o gargalo da memória com *delay* de 8 ciclos, visto que a cache irá funcionar com borda invertida, logo irá possuir tempo 0 para retorno das informações.

A cache foi modelada com as instruções fornecidas (4 linhas e blocos de 8 palavras), logo o endereçamento possuirá 2 bits para linha e 3 bits para bloco, além de 27 bits de tag. Na implementação desenvolvida, optou-se por utilizar apenas 25 bits para a tag, uma vez que o endereçamento da memória no Mars ocorre em incrementos de 4 bits. Dessa forma, 5 bits são destinados ao endereçamento do bloco. Contudo, apenas 3 bits são necessários para endereçar o bloco da cache, resultando na subutilização de 2 bits mais significativos. Essa escolha se deu em virtude das características específicas do ambiente de desenvolvimento Mars. Ao utilizar apenas os 3 bits essenciais para endereçar o bloco da cache, os 2 bits restantes não são plenamente aproveitados.

O controle entre leitura e escrita é feito a partir de uma máquina de estados de 4 estados, sendo eles:

1. starting_mp: Estado inicial, apenas bloqueia a cache e passa todas as informações diretamente pra a memória RAM para que ela seja inicializada com a memória de dados do programa assembly
2. idle: Estado de espera, aguarda qualquer tipo de instrução do tipo *write* (escrita) ou *read* (leitura)
3. write_mp: Estado de escrita na memória RAM, serve para fazer *write through* dos dados para a memoria RAM (ativado sempre que ouver operação de escrita na cache).
4. read_mp: Estado de leitura da memória RAM, serve para preencher a linha de cache com os dados existentes na RAM no caso de miss da cache.

além disso, a máquina de estados controla a saída dos sinais de escrita e leitura para RAM, assim como o sinal de halt para o processador e os barramentos de conexão processador-cache e cache-RAM.

5 Problemas

Apenas 3 problemas foram constatados no código, entretanto nenhum deles realmente afetou de forma severa o resultado final dos códigos e do tempo de execução, e, visto que não conseguimos resolver, acabamos por apenas relatar eles aqui. Os problemas encontrados foram:

1. *Undefined* em alguns pontos da wave: Alguns "undefineds" acabaram aparecendo na wave, entretanto não conseguimos resolver, acreditamos que possa ser algum descuido com o barramento ou algum sinal não inicializado corretamente.
2. *Mem 0* com undefined: após a escrita correta dos valores da *mem 0*, ela acaba recebendo undefined, entretando isso só ocorre com esta parte da memória RAM em específico e não conseguimos indentificar exatamente o motivo pelo qual isto ocorre
3. *bubble sort* coloca valores incorretos na memória: no algoritmo *bubble sort*, os valores finais de ordenação não são os esperados, mas sim o valor do maior número do vetor. isto não ocorre para a MP-0

[*] lukas.e@edu.pucrs.br
[†] g.menezes@edu.pucrs.br
[‡] Emanuel.Barros@edu.pucrs.br

Pela nossa falta de conhecimento em VHDL não conseguimos solucionar os problemas indicados, mas indicamos para que as possíveis situações estejam bem definidas.

6 Graficos

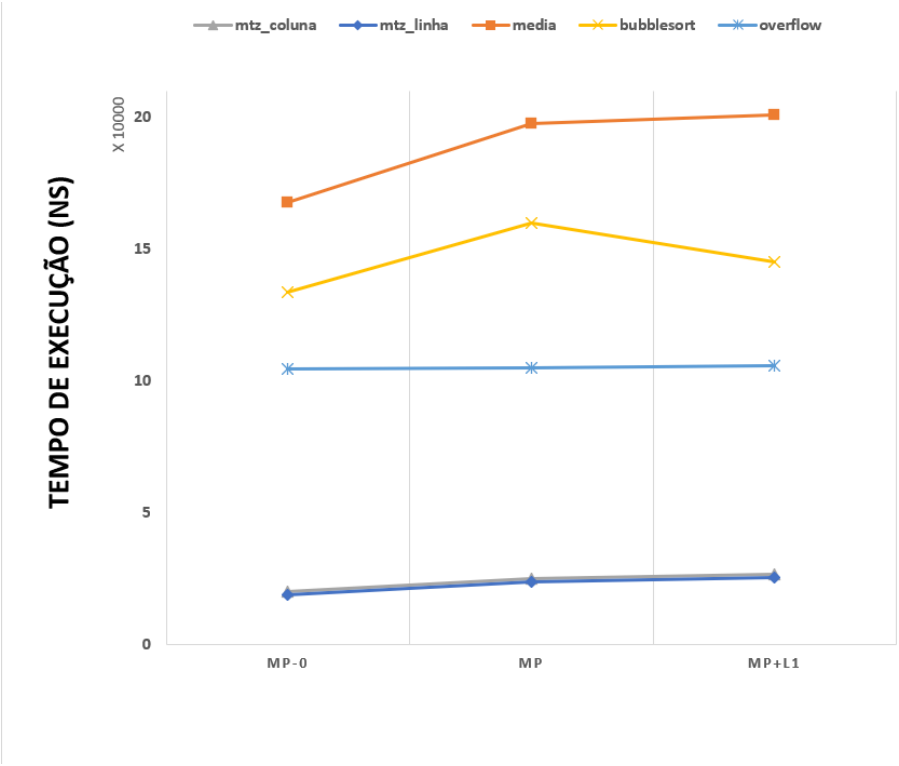


Figure 1: Gráfico com o tempo de execução para cada modelo

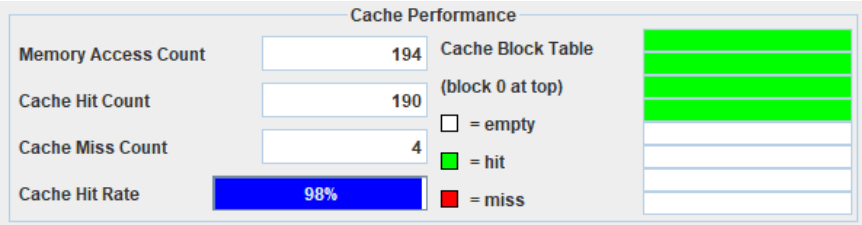


Figure 2: Performance da cache do Mars para o *bubble sort*

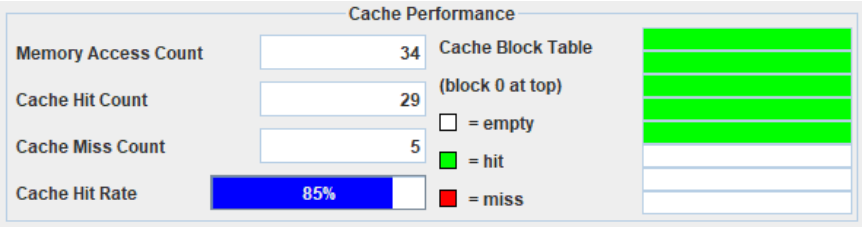


Figure 3: Performance da cache do Mars para a leitura por linha

7 Conclusão

Portanto, é possível concluir que o desenvolvimento de uma cache pode acelerar os processos, tanto de forma teórica, quanto prática. Entretanto, é necessária uma modelagem mais complexa do que a que foi desenvolvida neste trabalho. Logo, este processo exige um maior conhecimento, tanto de VHDL quanto de controle e otimização da cache, para que assim seja possível obter resultados melhores e mais aplicáveis.